Project Title

Mini Search Engine / Information-Retrieval System

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by

B. Deepak - 2520080035

A. Lakshmi Sai Teja - 2520080005

Team Number: 2

Section: 11

Under the guidance of

Mrs. Ch. Anitha

Asst. Professor, CS&IT



Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



DECLARATION

The Mini Project Report entitled "**Mini Search Engine / Information-Retrieval System**" is a record of Bonafide work of **B. Deepak – 2520080035, A. Lakshmi Sai Teja – 2520080005** submitted in partial fulfillment for the award of B. Tech in Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

B. Deepak – 2520080035

A. Lakshmi Sai Teja – 2520080005

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the mini project based report entitled "**Mini Search Engine / Information-Retrieval System**" is a bonafide work done and submitted by **B. Deepak – 2520080035, A. Lakshmi Sai Teja – 2520080005** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science and Information Technology, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success of this project would not have been possible but for the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who have assisted us in one way or the other in the completion of our project.

Our greatest appreciation goes to our guide, Mrs. Ch. Anitha, Assistant Professor, Department of Computer Science and Information Technology, whose tremendous support, encouragement and guidance made it possible to complete this project.

We express our gratitude to **Dr. Khaja Shareef**, Head of the Department of Computer Science and Information Technology, for providing us with adequate facilities, ways and means by which we were able to complete this project-based Lab.

We thank all the members of the teaching and non-teaching staff, and also those who have assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

B. Deepak – 2520080035

A. Lakshmi Sai Teja – 2520080005

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7
3.	Literature survey	8
4.	Problem Statement and Objectives	10
5.	Hardware and Software requirements	11
6.	Implementation	12
7.	Results & Discussion	19
8.	Conclusion & Future Scope	22
9.	References	23

1. ABSTRACT

This project presents the design and implementation of a custom Mini Search Engine / Information-Retrieval System built from first principles, without relying on any external search library. The system indexes a document corpus and supports four primary query classes: exact keyword matching, phrase and substring retrieval, fuzzy string matching, and document-similarity ranking.

To achieve this, the architecture uses hand-implemented data structures and algorithms: an inverted index for fast keyword look-ups, a suffix array for substring and phrase queries, dynamic programming for the Levenshtein distance used in fuzzy search, and TF-IDF vectorisation with cosine similarity for relevance ranking. The report justifies each algorithmic choice and derives the asymptotic time and space complexity of the indexing, querying and scaling phases.

The system was implemented in Java and tested on a ten-document corpus. It returned correct results for all four query classes; for example, the misspelt query “levenstein” was matched to “levenshtein” at edit distance 1, and documents D9 and D2 were ranked highest for the query “hash table keys”. Overall, the work gives a clear view of the mechanical design, algorithmic foundations and performance trade-offs found in modern search systems.

Team Name: Team 02

Team Members:

1. B. Deepak_2520080035
2. A. Lakshmi Sai Teja_2520080005

2. INTRODUCTION

2.1 Background

Search engines are the main way in which people find information in large collections of documents, from web pages and e-mails to research papers and source code. A search engine must answer a user query in a fraction of a second even when the collection holds millions of documents, so a naive scan of every document for every query is not acceptable. Information retrieval (IR) is the field that studies the data structures and algorithms which make this possible [1]. Production systems such as Apache Lucene, Solr and Elasticsearch hide these mechanisms inside large libraries, so a student who only uses them learns how to call a search API but not how a search actually works.

2.2 Motivation

Real users do not type only exact, correctly spelt keywords. They search for a phrase or a fragment of a word, they make spelling mistakes, and they expect the most relevant documents to appear first. Each of these needs is served by a different data structure or algorithm, and each has its own time and space trade-offs. This project was chosen because it brings together, in one working system, several of the topics of the Data Structures and Algorithms course: hashing, sorting, binary search, dynamic programming, heaps and vector-space ranking.

2.3 Proposed Approach

This project builds a Mini Search Engine / Information-Retrieval System from first principles, without using any external search library. The documents are tokenised and indexed once, and the following four query classes are then answered:

- **Exact keyword matching** using an inverted index that maps every term to the documents (and positions) in which it occurs.
- **Phrase and substring retrieval** using a suffix array of the whole corpus, searched by binary search.
- **Fuzzy string matching** using the Levenshtein edit distance, computed by dynamic programming, to find indexed terms close to a misspelt query.
- **Document-similarity ranking** using TF-IDF vectors and cosine similarity, with a min-heap to select the top-k results.

The report is organised as follows: Chapter 3 reviews related work, Chapter 4 states the problem and objectives, Chapter 5 lists the requirements, Chapter 6 explains the implementation together with its complexity analysis, Chapter 7 presents the results, Chapter 8 gives the conclusion and future scope, and Chapter 9 lists the references.

3. LITERATURE SURVEY

Text retrieval has been studied for more than fifty years. The following works and systems were studied while designing this search engine.

3.1 Introduction to Information Retrieval [1]. Manning, Raghavan and Schütze describe the complete architecture of a search engine: tokenisation, the inverted index, Boolean retrieval, tolerant retrieval for misspelt queries, and ranked retrieval with the vector-space model. Their book is the main reference for the overall structure of our system, and our four query classes follow the order in which it introduces these ideas.

3.2 Inverted files for text search engines [2]. Zobel and Moffat survey how inverted files are built, stored and compressed, and show that they give the best trade-off between query speed and space for keyword search over large collections. We store positions along with document identifiers in the same way, but keep the index in main memory without compression because our corpus is small.

3.3 Suffix arrays [3], [4]. Manber and Myers introduced the suffix array as a compact alternative to the suffix tree: a sorted array of the starting positions of all suffixes of a text. Any substring can be found by binary search in O(m log n) time, where m is the pattern length and n the text length. Gusfield's book explains how substring search is reduced to a range search over the sorted suffixes, which is exactly how our substring module works.

3.4 Edit distance [5], [6]. Levenshtein defined the minimum number of insertions, deletions and substitutions needed to turn one string into another, and Wagner and Fischer gave the dynamic-programming algorithm that computes it in O(mn) time. Tolerant retrieval uses this distance to match a misspelt query term against the vocabulary of the index, and we adopt the same approach.

3.5 TF-IDF term weighting [7], [8]. Spärck Jones showed that a term is a better discriminator when it occurs in few documents, which led to inverse document frequency (IDF). Salton and Buckley compared term-weighting schemes and established TF-IDF weighting with cosine-normalised vectors as a strong baseline. Our ranking module follows this scheme.

3.6 Apache Lucene [10]. Lucene is the open-source Java search library on which many search products are built. It uses a compressed inverted index, supports wildcard and edit-distance (fuzzy) queries, and ranks results with BM25, a member of the TF-IDF family. It is a powerful tool, but it is used as a black box; our project re-creates the underlying ideas by hand.

3.7 Elasticsearch [11]. Elasticsearch is a distributed search service built on top of Lucene. It adds sharding, replication and a REST interface, and offers fuzzy queries based on edit distance. It shows how an index is scaled across many machines, which we discuss as future scope.

3.8 Apache Solr [12]. Solr is another Lucene-based search platform with faceting, highlighting and a fuzzy-search operator. Like Elasticsearch it is a ready-made product, so it does not expose the individual data structures and their complexities that this project sets out to study.

3.9 Comparison of Existing Work with the Proposed System

Table 3.1 compares the systems studied above with the proposed search engine.

System	Core structure	Substring queries	Fuzzy queries	Ranking	Built from scratch
Lucene [10]	Compressed inverted index	Wildcard / n-gram	Edit-distance automata	BM25 (TF-IDF family)	No (library)
Elasticsearch [11]	Distributed Lucene indices	Wildcard / n-gram	Edit-distance fuzzy query	BM25 by default	No (service)
Solr [12]	Lucene-based inverted index	Wildcard / n-gram	Edit-distance (~) operator	BM25 by default	No (platform)
Proposed system	Inverted index + suffix array	Suffix array (exact substring)	DP Levenshtein distance	TF-IDF + cosine	Yes (all hand-written)

Table 3.1: Comparison of existing systems with the proposed search engine

3.10 Summary and Research Gap

Industrial search libraries are fast and feature-rich, but they hide the data structures and algorithms inside and are too large to read in a course project. Textbooks describe each technique separately. The present project fills this gap by implementing the inverted index, suffix array, dynamic-programming edit distance and TF-IDF cosine ranking together, in a small and readable Java code base, and by justifying the time and space complexity of every phase.

4. PROBLEM STATEMENT AND OBJECTIVES

Problem Statement: Scanning every document for every query is too slow, and a plain keyword lookup cannot handle phrases, word fragments, spelling mistakes or relevance ranking.

Problem Description: A search system must index a document corpus once and then answer exact, substring, fuzzy and similarity queries quickly, with every algorithm built by hand and its complexity justified.

Objectives:

- To design and build a search engine from first principles, without using any external search library.
- To index a document corpus with an inverted index so that exact keyword and multi-keyword (AND) queries are answered in near constant time per term.
- To support phrase and substring retrieval with a suffix array and binary search.
- To support fuzzy matching of misspelt terms using the Levenshtein distance computed by dynamic programming.
- To rank documents by relevance and similarity using TF-IDF vectorisation, cosine similarity and a top-k min-heap.
- To analyse and justify the asymptotic time and space complexity of the indexing, querying and scaling phases.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

- Processor: Intel Core i3 / AMD Ryzen 3 or higher (dual-core, 2.0 GHz or above)
- RAM: 4 GB minimum (8 GB recommended)
- Storage: at least 500 MB of free disk space for the project and the document corpus
- Display: 1366 × 768 resolution or higher

Software Requirements:

- Operating System: Windows 10 / 11 (Linux and macOS are also supported)
- Programming Language: Java (JDK 17 or later) for the indexing and query engine; the code in this report was compiled and run on JDK 21
- Data: plain-text documents forming the corpus
- IDE / Tools: IntelliJ IDEA or VS Code, with Git for version control
- Libraries: only the Java standard library (java.util); no external search library is used

6. IMPLEMENTATION

6.1 System Architecture and Workflow

The search engine works in two phases. In the indexing phase every document is tokenised and three structures are built: an inverted index, a suffix array and a set of TF-IDF vectors. In the query phase the user query is routed to the structure that suits its type, and the result is returned as a list of document identifiers (and scores for ranked queries).

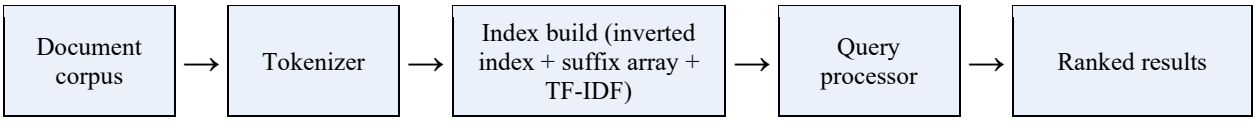


Fig. 6.1: Processing pipeline of the search engine

- **Tokenizer:** converts text to lower case and splits it into alphanumeric tokens.
- **Inverted index:** maps each term to the documents and token positions where it occurs; answers keyword and AND queries.
- **Suffix array:** sorts all suffixes of the concatenated corpus; answers phrase and substring queries by binary search.
- **Fuzzy search:** computes the Levenshtein distance between the query and the index vocabulary to find close terms.
- **TF-IDF ranker:** builds a weight vector for each document and ranks documents by cosine similarity.

6.2 Data Structures and Algorithms Used

Component	Data structure / Algorithm	Purpose	Complexity
Tokenizer	Single character scan	Splits text into terms	O(n) time
Inverted index	HashMap of term → TreeMap(doc → position list)	Keyword and AND queries	O(1) average lookup
Suffix array	Prefix-doubling sort + binary search	Phrase and substring queries	Build $O(n \log^2 n)$; query $O(m \log n)$
Fuzzy search	Dynamic programming (Levenshtein, two rows)	Match misspelt terms	$O(m \cdot L)$ time, $O(L)$ space per term
TF-IDF vectors	HashMap vectors + cached norms	Weight terms per document	O(T) build
Top-k selection	Min-heap (PriorityQueue) of size k	Best k results	O(D log k)

Table 6.1: Data structures and algorithms

6.3 Module 1: Tokenizer and Inverted Index (Tokenizer.java, InvertedIndex.java)

The tokenizer lower-cases the text and cuts it at every non-alphanumeric character. The inverted index stores, for every term, a map from document identifier to the sorted list of positions of the term. A keyword query is a single hash lookup, and an AND query intersects the posting sets of its terms.

```
import java.util.*;
```

```
public final class Tokenizer {
    /** Lower-cases the text and splits it into alphanumeric tokens. */
    public static List<String> tokenize(String text) {
        List<String> out = new ArrayList<>();
        StringBuilder sb = new StringBuilder();
        for (char c : text.toLowerCase().toCharArray()) {
            if (Character.isLetterOrDigit(c)) sb.append(c);
            else if (sb.length() > 0) { out.add(sb.toString()); sb.setLength(0); }
        }
        if (sb.length() > 0) out.add(sb.toString());
        return out;
    }
}
```

```
import java.util.*;

public final class InvertedIndex {
    // term -> (docId -> sorted list of token positions)
    private final Map<String, Map<Integer, List<Integer>>> index = new HashMap<>();

    public void addDocument(int docId, String text) {
        List<String> tokens = Tokenizer.tokenize(text);
        for (int pos = 0; pos < tokens.size(); pos++)
            index.computeIfAbsent(tokens.get(pos), k -> new TreeMap<>())
                 .computeIfAbsent(docId, k -> new ArrayList<>()).add(pos);
    }

    /** Documents containing the term: O(1) average lookup. */
    public Set<Integer> search(String term) {
        Map<Integer, List<Integer>> p = index.get(term.toLowerCase());
        return p == null ? new TreeSet<>() : p.keySet();
    }

    /** AND query: intersect the posting lists of all terms. */
    public Set<Integer> searchAll(String... terms) {
        Set<Integer> result = null;
        for (String t : terms) {
            Set<Integer> s = search(t);
            if (result == null) result = new TreeSet<>(s); else result.retainAll(s);
        }
        return result == null ? new TreeSet<>() : result;
    }

    public int termFrequency(String term, int docId) {
        Map<Integer, List<Integer>> p = index.get(term);
        return (p == null || !p.containsKey(docId)) ? 0 : p.get(docId).size();
    }
    public int documentFrequency(String term) { return search(term).size(); }
    public Set<String> vocabulary() { return index.keySet(); }
}
```

6.4 Module 2: Suffix Array for Substring and Phrase Search (SuffixArray.java)

All documents are joined into one text, separated by a character that cannot occur in a document. The suffixes are sorted by prefix doubling: after round k every suffix is ranked by its first 2k characters. A pattern is found by two binary searches that give the range of suffixes starting with the pattern; the start offset of each suffix is mapped back to its document by a third binary search.

```
import java.util.*;

public final class SuffixArray {
```

```
    private final String text;
    private final int[] sa;           // sorted suffix start positions
    private final int[] docStart;     // start offset of each document in text

    public SuffixArray(List<String> docs) {
        StringBuilder sb = new StringBuilder();
        docStart = new int[docs.size()];
        for (int i = 0; i < docs.size(); i++) {
            docStart[i] = sb.length();
            sb.append(docs.get(i).toLowerCase()).append('\u0001'); // separator
        }
        text = sb.toString();
        sa = build(text);
    }

    /** Prefix doubling: O(n log^2 n) time, O(n) space. */
    private static int[] build(String s) {
        int n = s.length();
        Integer[] idx = new Integer[n];
        int[] rank = new int[n], tmp = new int[n];
        for (int i = 0; i < n; i++) { idx[i] = i; rank[i] = s.charAt(i); }
        for (int k = 1; ; k <<= 1) {
            final int kk = k;
            final int[] r = rank;
            Comparator<Integer> cmp = (a, b) -> {
                if (r[a] != r[b]) return Integer.compare(r[a], r[b]);
                int ra = a + kk < n ? r[a + kk] : -1, rb = b + kk < n ? r[b + kk] : -1;
                return Integer.compare(ra, rb);
            };
            Arrays.sort(idx, cmp);
            tmp[idx[0]] = 0;
            for (int i = 1; i < n; i++)
                tmp[idx[i]] = tmp[idx[i - 1]] + (cmp.compare(idx[i - 1], idx[i]) < 0 ? 1 : 0);
            System.arraycopy(tmp, 0, rank, 0, n);
            if (rank[idx[n - 1]] == n - 1) break;
        }
        int[] res = new int[n];
        for (int i = 0; i < n; i++) res[i] = idx[i];
        return res;
    }

    /** Binary search for the range of suffixes starting with the pattern: O(m log n). */
    public Set<Integer> search(String pattern) {
        String p = pattern.toLowerCase();
        int lo = lowerBound(p, false), hi = lowerBound(p, true);
        Set<Integer> docs = new TreeSet<>();
        for (int i = lo; i < hi; i++) docs.add(docOf(sa[i]));
        return docs;
    }

    private int lowerBound(String p, boolean upper) {
        int lo = 0, hi = sa.length;
        while (lo < hi) {
            int mid = (lo + hi) >>> 1;
            int end = Math.min(sa[mid] + p.length(), text.length());
            int c = text.substring(sa[mid], end).compareTo(p);
            if (c < 0 || (upper && c == 0)) lo = mid + 1; else hi = mid;
        }
        return lo;
    }

    private int docOf(int pos) {
        int d = Arrays.binarySearch(docStart, pos);
        return d >= 0 ? d : -d - 2;
    }
}
```

6.5 Module 3: Fuzzy Search with Levenshtein Distance (FuzzySearch.java)

The edit distance is computed with the dynamic-programming recurrence cur[j] = min(cur[j-1]+1, prev[j]+1, prev[j-1]+cost). Only two rows of the table are kept, so the space is O(n) instead of O(mn). Terms whose length differs from the query by more than the allowed distance are skipped without computing the distance.

```
import java.util.*;

public final class FuzzySearch {
    /** Levenshtein edit distance using two rolling rows: O(m*n) time, O(n) space. */
    public static int distance(String a, String b) {
        int[] prev = new int[b.length() + 1], cur = new int[b.length() + 1];
        for (int j = 0; j <= b.length(); j++) prev[j] = j;
        for (int i = 1; i <= a.length(); i++) {
            cur[0] = i;
            for (int j = 1; j <= b.length(); j++) {
                int cost = a.charAt(i - 1) == b.charAt(j - 1) ? 0 : 1;
                cur[j] = Math.min(Math.min(cur[j - 1] + 1, prev[j] + 1), prev[j - 1] + cost);
            }
            int[] t = prev; prev = cur; cur = t;
        }
        return prev[b.length()];
    }

    /** Vocabulary terms within maxDist edits of the query, closest first. */
    public static List<String> closeTerms(String query, Set<String> vocab, int maxDist) {
        List<String> hits = new ArrayList<>();
        for (String t : vocab)
            if (Math.abs(t.length() - query.length()) <= maxDist
                    && distance(query.toLowerCase(), t) <= maxDist) hits.add(t);
        hits.sort(Comparator.comparingInt((String t) -> distance(query.toLowerCase(), t))
                            .thenComparing(Comparator.naturalOrder()));
        return hits;
    }
}
```

6.6 Module 4: TF-IDF Vectorisation and Cosine Ranking (TfIdfRanker.java)

Every term t in a document d receives the weight w(t, d) = (1 + ln tf) × ln(N / df), where tf is the number of occurrences of t in d, df the number of documents containing t, and N the total number of documents. The similarity of a query q and a document d is the cosine of the angle between their vectors, cos(q, d) = (q · d) / (|q| |d|). A min-heap of size k keeps the best k documents while all documents are scored.

```
import java.util.*;

public final class TfIdfRanker {
    private final InvertedIndex index;
    private final int numDocs;
    private final List<Map<String, Double>> vectors = new ArrayList<>();
    private final double[] norms;

    public TfIdfRanker(InvertedIndex index, int numDocs) {
        this.index = index; this.numDocs = numDocs; this.norms = new double[numDocs];
        for (int d = 0; d < numDocs; d++) vectors.add(new HashMap<>());
        for (String t : index.vocabulary()) {
            double idf = Math.log((double) numDocs / index.documentFrequency(t));
            for (int d : index.search(t)) {
                double w = (1 + Math.log(index.termFrequency(t, d))) * idf;
                vectors.get(d).put(t, w);
```

```
                norms[d] += w * w;
            }
        }
        for (int d = 0; d < numDocs; d++) norms[d] = Math.sqrt(norms[d]);
    }

    /** Cosine similarity between a document and another document vector. */
    private double cosine(Map<String, Double> q, double qn, int d) {
        if (qn == 0 || norms[d] == 0) return 0;
        double dot = 0;
        for (Map.Entry<String, Double> e : q.entrySet())
            dot += e.getValue() * vectors.get(d).getOrDefault(e.getKey(), 0.0);
        return dot / (qn * norms[d]);
    }

    /** Rank all documents against a free-text query; top-k by score. */
    public List<Map.Entry<Integer, Double>> rank(String query, int k) {
        Map<String, Double> q = new HashMap<>();
        for (String t : Tokenizer.tokenize(query)) {
            int df = index.documentFrequency(t);
            if (df == 0) continue;
            q.merge(t, Math.log((double) numDocs / df) + 0.0001, Double::sum);
        }
        double qn = Math.sqrt(q.values().stream().mapToDouble(v -> v * v).sum());
        return topK(d -> cosine(q, qn, d), k, -1);
    }

    /** Documents most similar to document docId ("more like this"). */
    public List<Map.Entry<Integer, Double>> similarTo(int docId, int k) {
        Map<String, Double> q = vectors.get(docId);
        return topK(d -> cosine(q, norms[docId], d), k, docId);
    }

    private List<Map.Entry<Integer, Double>> topK(
            java.util.function.IntToDoubleFunction f, int k, int skip) {
        PriorityQueue<Map.Entry<Integer, Double>> heap =
            new PriorityQueue<>(Map.Entry.comparingByValue()); // min-heap of size k
        for (int d = 0; d < numDocs; d++) {
            if (d == skip) continue;
            double s = f.applyAsDouble(d);
            if (s <= 0) continue;
            heap.add(new AbstractMap.SimpleEntry<>(d, s));
            if (heap.size() > k) heap.poll();
        }
        List<Map.Entry<Integer, Double>> out = new ArrayList<>(heap);
        out.sort((a, b) -> Double.compare(b.getValue(), a.getValue()));
        return out;
    }
}
```

6.7 Complexity Analysis

Let D be the number of documents, T the total number of tokens, n the total number of characters in the corpus, V the vocabulary size, L the length of the longest term, m the length of the query pattern, q the number of query terms and k the number of results returned.

Phase	Operation	Time	Space
Indexing	Build inverted index	O(T) average	O(T)
Indexing	Build suffix array	O(n log² n)	O(n)
Indexing	Build TF-IDF vectors	O(T)	O(distinct term–document pairs)

Phase	Operation	Time	Space
Querying	Keyword lookup	O(1) average + O(df) to list	O(1) extra
Querying	AND query (terms t1…tq)	O(Σ dfi · log D)	O(smallest df)
Querying	Substring / phrase	O(m log n + occ · log D)	O(occ)
Querying	Fuzzy match of one term	O(V · m · L) worst, less with length filter	O(L)
Querying	Ranked retrieval	O(D · q + D log k)	O(k)

Table 6.2: Time and space complexity of each phase

Justification. The inverted index inserts every token once into a hash map, which gives O(T) average time. The suffix array needs O(log n) doubling rounds, each of which sorts n suffixes in O(n log n) comparisons, giving O(n log² n); a query then needs O(log n) steps of an O(m) comparison. The Levenshtein table has m × L cells, and it must be evaluated against every vocabulary term in the worst case. Ranking evaluates one dot product of at most q terms for each of the D documents, and the heap adds O(log k) per document.

Scaling. Index build time grows almost linearly with the corpus for the inverted index and TF-IDF vectors, so these scale well. The suffix array is the most expensive part, because it stores n integers and its construction is super-linear; for very large corpora it would be built per shard with the linear-time SA-IS algorithm. The cost of fuzzy search grows with the vocabulary size V, which grows slowly with the corpus, and the cost of ranking grows linearly with D unless candidates are first restricted through the inverted index. Splitting the documents across shards and merging the top-k lists is the standard way to scale beyond a single machine.

6.8 Verification

The modules were compiled and run on a ten-document corpus (D1 to D10, listed in Table 7.1). Every result was checked by hand against the corpus: the word “hash” appears only in D2 and D9, the word “sorted” only in D1 and D6, the string “th in” only in “path in a graph” of D3, and the misspelt term “levenstein” is exactly one insertion away from “levenshtein” in D5. The outputs are shown in Chapter 7.

7. RESULTS & DISCUSSION

The output of the running application and a short explanation of each result are given below. The test corpus is listed in Table 7.1.

Doc	Text
D1	Binary search trees store keys in sorted order and support fast search, insertion and deletion.
D2	A hash table maps keys to values using a hash function and gives constant time lookup on average.
D3	Dijkstra’s algorithm finds the shortest path in a graph with non-negative edge weights.
D4	Dynamic programming solves problems by combining solutions of overlapping subproblems.
D5	The Levenshtein distance counts the minimum edits needed to turn one string into another.
D6	A suffix array lists all suffixes of a string in sorted order and supports fast substring search.
D7	Graph traversal uses breadth first search or depth first search to visit every vertex.
D8	An inverted index maps each word to the documents that contain it and powers search engines.
D9	Hash functions spread keys evenly across buckets to reduce collisions in a hash table.
D10	TF-IDF weighs a word by its frequency in a document and its rarity across the whole corpus.

Table 7.1: Test corpus used for the experiments

7.1 Console Output

```
Documents: 10, vocabulary size: 104

[1] Keyword search: "hash"
    -> [D2, D9]
    Keyword AND search: "hash" AND "table"
    -> [D2, D9]

[2] Substring search (suffix array): "sort"
    -> [D1, D6]
    Phrase search: "sorted order"
    -> [D1, D6]
    Cross-word substring search: "th in"
    -> [D3]

[3] Fuzzy search: "levenstein" (max edit distance 2)
    Edit distance(levenstein, levenshtein) = 1
    Close terms: [levenshtein]
    -> [D5]
    Fuzzy search: "serch" (max edit distance 1)
    Close terms: [search]

[4] Ranked retrieval (TF-IDF + cosine): "hash table keys"
    D9  score = 0.4757
    D2  score = 0.4087
    D1  score = 0.0836
    Documents most similar to D6:
    D1  cosine = 0.2135
    D4  cosine = 0.0515
    D5  cosine = 0.0464
```

Fig. 7.1: Output of the search engine for the four query classes

7.2 Explanation of the Results

Keyword search. The inverted index returns D2 and D9 for “hash”, and the AND query “hash” AND “table” returns the same two documents because both contain both words.

Substring and phrase search. The suffix array finds “sort” (inside “sorted”) and the phrase “sorted order” in D1 and D6. The cross-word fragment “th in” is found only in D3 (“path in a graph”), which a word-based index could not find.

Fuzzy search. The misspelt query “levenstein” is one edit away from “levenshtein”, so D5 is returned; “serch” is one edit away from “search”.

Ranked retrieval. For “hash table keys” the documents D9 (0.4757) and D2 (0.4087) score far above D1 (0.0836), because they contain the rare terms “hash” and “table”. For D6 (suffix array), the most similar document is D1 (cosine 0.2135), which shares the terms “sorted”, “order” and “search”.

7.3 Summary of Results

Measure	Value
Documents indexed	10
Vocabulary size (distinct terms)	104
Keyword query “hash”	D2, D9
Phrase query “sorted order”	D1, D6
Fuzzy query “levenstein” (distance 1)	levenshtein → D5
Best match for “hash table keys”	D9 (score 0.4757), then D2 (0.4087)
Most similar document to D6	D1 (cosine 0.2135)

Table 7.2: Summary of results

7.4 Discussion

The results show that each query class needs its own structure. The inverted index is the fastest for whole words but cannot see inside a word or across word boundaries, which the suffix array can. Neither handles a spelling mistake, which is why the edit distance is applied to the vocabulary and the closest term is then looked up in the inverted index. Finally, TF-IDF with cosine similarity orders the matching documents by relevance, giving more weight to rare terms such as “hash” than to common ones.

The main limitations are that the corpus is small, so no timing benchmark is claimed; that the whole index is held in memory; that no stop-word removal or stemming is applied (so “sorted” and “sort” are different terms for the inverted index); and that fuzzy search compares the query with every vocabulary term.

8. CONCLUSION & FUTURE SCOPE

8.1 Conclusion

The Mini Search Engine / Information-Retrieval System meets its objectives. It was built entirely from first principles in Java, without any external search library, and supports exact keyword, phrase and substring, fuzzy, and similarity-ranked queries. On the ten-document test corpus every query returned the expected documents: “hash” was found in D2 and D9, “sorted order” in D1 and D6, the misspelt “levenstein” was matched to D5 at edit distance 1, and D9 and D2 were ranked first for “hash table keys”.

The project also gave practical experience of the data structures and algorithms studied in the course: hashing for the inverted index, sorting and binary search for the suffix array, dynamic programming for the edit distance, and a heap for top-k selection. The complexity analysis shows the trade-offs of each choice, such as the fast lookups of the inverted index against the high memory and build cost of the suffix array.

8.2 Future Scope

Near term

- Add stop-word removal and stemming so that “sort” and “sorted” share an index entry.
- Use the stored word positions to answer phrase queries from the inverted index and compare with the suffix array.
- Add BM25 scoring and compare its ranking with TF-IDF.

Medium term

- Speed up fuzzy search with a trie or a BK-tree, or with n-gram candidate filtering, instead of scanning the whole vocabulary.
- Build the suffix array with the linear-time SA-IS algorithm and store the index on disk with compression.
- Add a command-line or web interface with result highlighting and benchmarks on larger corpora.

Long term

- Shard the index across several machines and merge the top-k results of each shard.
- Support incremental updates so that new documents can be added without rebuilding the index.
- Add semantic search with learned text embeddings alongside lexical matching.

9. REFERENCES

[1] C. D. Manning, P. Raghavan and H. Schütze, Introduction to Information Retrieval, Cambridge University Press, 2008.

[2] J. Zobel and A. Moffat, "Inverted Files for Text Search Engines," ACM Computing Surveys, vol. 38, no. 2, article 6, 2006.

[3] U. Manber and G. Myers, "Suffix Arrays: A New Method for On-Line String Searches," SIAM Journal on Computing, vol. 22, no. 5, pp. 935–948, 1993.

[4] D. Gusfield, Algorithms on Strings, Trees, and Sequences, Cambridge University Press, 1997.

[5] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," Soviet Physics Doklady, vol. 10, no. 8, pp. 707–710, 1966.

[6] R. A. Wagner and M. J. Fischer, "The String-to-String Correction Problem," Journal of the ACM, vol. 21, no. 1, pp. 168–173, 1974.

[7] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," Information Processing & Management, vol. 24, no. 5, pp. 513–523, 1988.

[8] K. Spärck Jones, "A statistical interpretation of term specificity and its application in retrieval," Journal of Documentation, vol. 28, no. 1, pp. 11–21, 1972.

[9] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th ed., MIT Press, 2022.

[10] Apache Lucene: https://lucene.apache.org

[11] Elasticsearch: https://www.elastic.co/elasticsearch

[12] Apache Solr: https://solr.apache.org

[13] Oracle, Java Platform, Standard Edition Documentation: https://docs.oracle.com/en/java/